Exercise 1

For each city i we first compute the prefix sum array of how many attractions can be visited in it, and store it in $\text{cities}[i]$. The value $\text{cities}[i][j]$ therefore corresponds to how many attractions you can visit in city i if you stay there for j days.

Next we define $\text{best}[i][j]$ to be a 2D-array consisting of the maximum value achievable after considering the first i cities using exactly j days. We have the following relations:

- Any case where 0 cities have been visited has best value 0;
- Any case where 0 days have been used has best value 0;
- For each city i and day budget j, we iterate over the number k of days spent exactly in city i and store the best

$$\text{best}[i][j] = \max_{0 \le k \le j} (\text{best}[i-1][j-k] + \text{cities}[i][k]).$$

The final answer is stored in $\text{best}[n][D]$.

Each prefix sum computation requires $\Theta(D)$ time, for a total of $\Theta(nD)$. To compute each $\text{best}[i][j]$ it takes $\Theta(j)$ time (which is at most D), yielding $O(D)$ for each tuple (i, j). Since $0 \le i \le n$ and $0 \le j \le D$ the bottom-up requires $O(nD^2)$ time. The overall time complexity is therefore $O(nD^2)$. The prefix sums are stored in a $n \times D$ matrix, the best values are stored in a $(n+1) \times (D+1)$ matrix. The overall space complexity is therefore $\Theta(nD)$.

Exercise 2

A possible first approach is trying to solve the problem like a **Longest Increasing Subsequence (LIS)**. If we impose the order

$$(b, d) < (b', d') \iff b < b', d < d'$$

and apply **LIS** we in fact get a suitable choice of topics, moreover if the beauty values of all topics are distinct this is also the best choice. However, when multiple topics share the same beauty but have different difficulties, a conflict can arise: the standard **LIS** implementation does not choose sensibly between topics with same beauty. To resolve this, we apply a greedy strategy: among all topics with the same beauty, we prefer the one with the smallest difficulty (provided it remains strictly greater than the difficulty of the previous topic in the sequence). Sorting the topics by *increasing beauty* first and *decreasing difficulty* second ensures that for any group of equal beauty, the higher-difficulty items appear first. Hence, the greedy nature of the **LIS** algorithm ensures optimality is preserved by picking the least difficult between the possible ones.

In practice, to implement this idea we can simply sort the topics by increasing beauty first and decreasing difficulty second. If we run **LIS** with the order

$$(b, d) < (b', d') \iff b < b' \vee (b = b' \wedge d < d')$$

the greedy nature of the algorithm is going to pick the topic between the plausible ones with least difficulty as we desire.

Sorting the topics can be done in $O(n \log n)$ time, and so is required to find the **LIS**. The overall time complexity is $O(n \log n)$. Sorting can be made in $O(n)$ space (depending on

the implementation, but this is usually enough) and **LIS** uses at most $O(n)$ additional space. Overall space complexity is $O(n)$.